

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Towards AI · [Follow publication](#)

AI

+

Artificial Intelligence

+

Deep Learning

+

LLM

+

The Secret Behind Fast LLM Inference: Unlocking the KV Cache

10 min read · Nov 13, 2025



Burak Degirmencioglu

Follow



Listen



Share

Large Language Models (LLMs) are incredibly powerful, but generating text with them can be surprisingly slow. This is because standard Transformer models, in a process called autoregressive generation, must re-process the entire history of the conversation for every single new word they produce.

This inefficiency is a major bottleneck. The solution is a critical optimization technique known as the **KV Cache**.

In this article, we will explore what the KV Cache is, how it dramatically speeds up LLM inference, the memory challenges it introduces, and the advanced strategies used to manage it effectively.

Open in app ↗

Sign up

Sign in

Medium



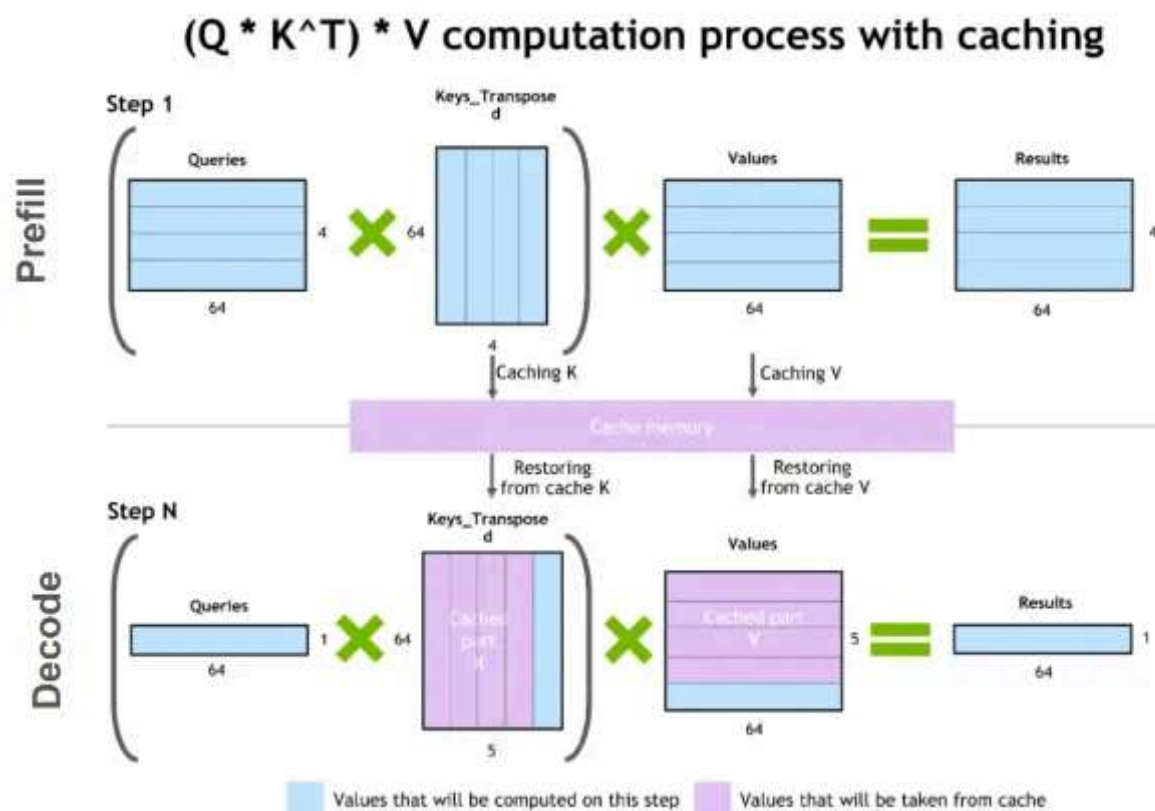
Search



dedicated space in high-speed GPU memory used to store the intermediate **Key (K)** and **Value (V)** vectors from the self-attention layers. In a Transformer, these K and V vectors represent the “lookup” information from previous tokens. By storing them, the model gains a memory of what’s been processed, avoiding redundant work.

But why is this “redundant work” such a big issue in the first place? Let’s look at what happens

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.



[source](#)

Why is Autoregressive Generation So Inefficient?

Without caching, standard autoregressive generation is extremely inefficient. To generate the 100th token in a sequence, the model must first process the 99 preceding tokens to compute their Key and Value states. To generate the 101st token, it must *again* process all 100 preceding tokens, including the 99 it *just* processed. This repetitive recalculation results in computational complexity that grows quadratically with the sequence length, making real-time generation for long contexts practically impossible.

This quadratic scaling is a massive performance killer. So, how does storing the Keys and Values fundamentally solve this problem?

How Does Caching Re-use Past Computations?

The **KV Cache** elegantly solves this by introducing statefulness. Instead of recomputing everything, the model stores the Key and Value vectors for all previous tokens in the cache. When it’s time to generate the next token, the model only performs the attention calculation for the *single new token*. This new K and V pair is

then simply concatenated to the existing K and V tensors stored in the cache, making the

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

The conceptual benefit is clear, but what does this mean in terms of real-world performance?

Feature	Standard Inference	KV Caching
Computation per Word	The model repeats the same calculations for every word.	The model reuses past calculations for faster results.
Memory Usage	Uses less memory at each step, but memory grows with longer texts.	Uses extra memory to store past information, but keeps things efficient.
Speed	Gets slower as the text gets longer because it repeats work.	Stays fast even with longer texts by avoiding repeated work.
Efficiency	High computational cost and slower response times.	Faster and more efficient since the model remembers past work.
Handling Long Texts	Struggles with long texts due to repeated calculations.	Perfect for long texts as it remembers past steps.

[source](#)

What Are the Key Benefits of Using a KV Cache?

The performance gains are transformative. By avoiding redundant calculations, the KV Cache changes the computational cost of each new token from a quadratic problem to a linear one. This dramatically lowers latency and significantly accelerates text generation, which is what makes conversational AI feel responsive. This efficiency is also what enables models to handle the very long context windows and extended conversations that are now common, vastly improving the overall compute efficiency and user experience.

We’ve established that the KV Cache is essential for performance. Now, let’s break down the precise mechanics of how it operates during inference.

What is the Two-Phase Process of KV Caching?

The caching mechanism is best understood as a two-phase process. First is the **Prefill** phase.

When you provide an input prompt (e.g., “The capital of France is”), the model processes all of these tokens in parallel. It calculates the Key and Value vectors for

every token in this initial prompt and stores them in the cache, finally generating the very first token.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Second is the **decode** phase.

This is the generation loop for all subsequent tokens. For each new step, only the *single* previously generated token (e.g., “Paris”) is fed into the model. The model calculates the K and V vectors for *only this one token*. It then retrieves the “The capital of France is” K/V vectors from the cache and appends the new “Paris” K/V vectors to them. Attention is then computed using this complete, updated set of Keys and Values to generate the next token. This loop repeats, adding one token to the cache at a time.

This process of appending K and V vectors implies a specific data structure. How is this cache actually organized within the model?

How is the KV Cache Structured Inside the Model?

The cache isn't one giant block of memory; it's highly structured. A separate KV cache is maintained for *every single Transformer layer* in the model. For a model like Llama 3 8B with 32 layers, this means there are 32 independent KV caches. Each of these per-layer caches accumulates or grows as new tokens are generated, with the tensors expanding along the sequence-length dimension at each decoding step.

Understanding the theory is one thing, but how would a developer actually implement this mechanism in code?

What Does the Core Logic for a KV Cache Look Like in Code?

When implementing this, a developer would typically modify the model's `MultiHeadAttention` module. This involves registering `cache_k` and `cache_v` as non-persistent buffers within the module, so they are not saved as part of the model's standard `state_dict`.

The `forward` pass of the attention layer is then updated to accept a `use_cache=True` flag. Inside the forward pass, conditional logic checks if the cache is populated. If the cache is empty (the prefill step), it is initialized with the computed keys and values.

On all subsequent steps (the decode steps), the newly computed keys and values for the single token are appended to the existing cache tensors using a concatenation

operation like `torch.cat`. This also requires carefully managing the attention mask to ensure it's correctly updated. To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy. Sensors.

This code handles the use of the cache, but what about managing its lifecycle?

How Do You Manage the Cache Between Different Prompts?

A critical part of cache management is handling its lifecycle. Since the cache stores the history of a *specific* generation sequence, it must be cleared before starting a new, independent sequence.

For instance, if a user finishes one conversation and starts a new one, the old K/V states are irrelevant and would pollute the new generation. This is typically handled by implementing a `reset_cache()` method on the model, which sets the `cache_k` and `cache_v` buffers back to an empty state, making the model ready for a fresh prompt.

We've just praised the KV Cache as a massive optimization, but it introduces its own significant, new problem.

How Does the KV Cache Become a Memory Bottleneck?

The KV cache's primary drawback is its enormous memory footprint. The cache must store a Key and Value vector, for every head, for every layer, for every token in the sequence, for every item in the batch. This means its size scales linearly with both the sequence length and the batch size.

For example, processing a large batch of requests or summarizing a very long document can cause the cache to consume gigabytes of precious GPU VRAM, often far more than the model weights themselves. This size can be calculated as:

$$(\text{Batch Size}) \times (\text{Sequence Length}) \times (\text{Num Layers}) \times (\text{Num Heads}) \times (\text{Head Dimension}) \times 2 \text{ (for K and V)} \times (\text{bytes per parameter})$$

When the cache grows this large, what is the practical consequence for the system?

What Happens When the Cache Overwhelms GPU Memory?

This massive memory consumption quickly becomes the primary bottleneck in the entire inference pipeline. It severely limits the maximum batch size or the maximum context window a model can handle before running out of memory.

Get Burak Degirmencioglu's stories in your inbox

Join Medium for free to get updates from this writer.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Subscribe

☐ Remember me for faster sign in

Consequently, the performance of LLM inference shifts from being **compute-bound** (limited by the speed of mathematical calculations) to being **memory-bandwidth bound** (limited by the speed at which data can be moved to and from the GPU memory).

This memory bottleneck is the central challenge of modern LLM inference. Naturally, an entire field of advanced strategies has emerged to combat it.

How Do Frameworks Like Hugging Face Optimize Cache Implementation?

Frameworks like Hugging Face `transformers` offer different cache implementations. The default is often `DynamicCache`, where the cache tensor grows as needed by appending new tokens. While flexible, this dynamic shape-changing prevents modern JIT compilers like `torch.compile` from applying powerful optimizations.

The alternative is `StaticCache`, which pre-allocates a cache for a fixed maximum sequence length. Although it may use more memory upfront, its static shape allows `torch.compile` to fully optimize the generation loop, leading to significant speedups.

But what if the cache is simply too large to fit in VRAM, even with static allocation?

What is KV Cache Offloading?

When the cache is too large for GPU VRAM, the next logical step is **KV Cache Offloading**.

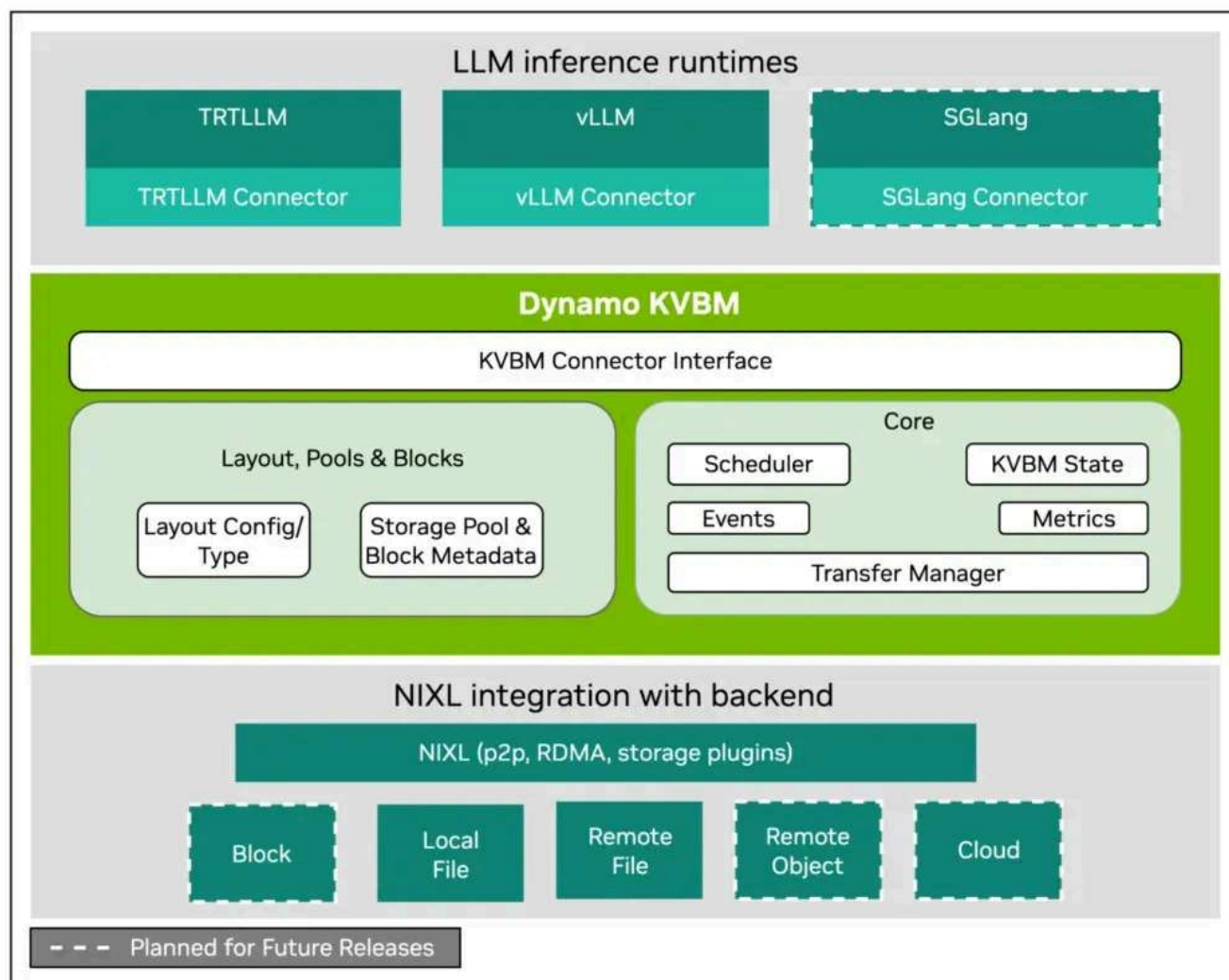
This technique involves intelligently moving inactive or less-used blocks of the KV cache from expensive, fast GPU VRAM to cheaper, more abundant storage, such as main system CPU RAM or even SSDs. This frees up GPU memory, allowing the system to process larger batches or much longer sequences than would otherwise be possible.

Technologies like the NVIDIA Dynamo KV Cache Manager implement this using sophisticated performan

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

g the

Offloading is great for a single GPU, but how do large-scale systems manage caches across an entire cluster?



[source](#)

How is the KV Cache Managed in a Distributed System?

In large, distributed inference systems, managing the cache becomes even more complex. The system must track which parts of which cache reside on which GPU. The NVIDIA Dynamo Smart Router, for example, uses a data structure like a Radix Tree to maintain a map of all KV cache block locations across a fleet of GPUs. When a new request arrives, this router can intelligently direct it to a GPU that already holds the necessary cache blocks, minimizing recomputation and network traffic. Systems like LMCache are also designed as high-performance management layers specifically for this distributed LLM inference challenge.

Whether *distributed or local*, a cache that grows indefinitely will eventually run out of space. To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

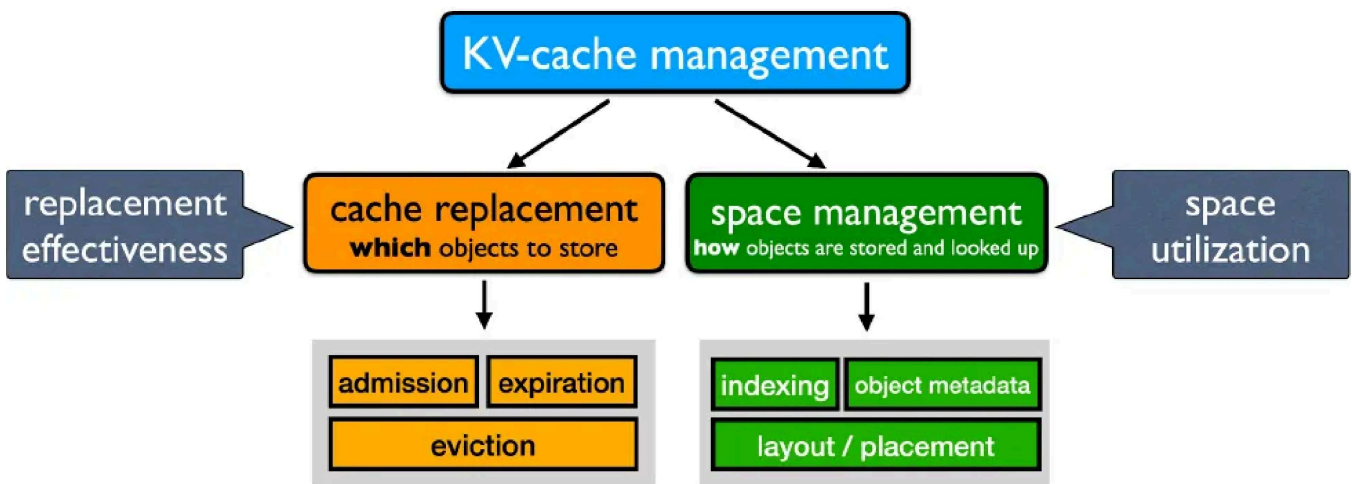
What Are the Strategies for Cache Eviction?

When a cache reaches its maximum size, the model must decide which tokens to “forget” a process called eviction. A simple “first-in, first-out” (FIFO) approach is often insufficient.

The **StreamingLLM** technique, for instance, uses a fixed-size cache that employs a sliding window approach. It cleverly retains the *initial tokens* (known as “attention sink tokens,” which are crucial for context) while also keeping the *most recent* tokens, discarding those in the middle.

Other systems like **PagedAttention** (popularized by vLLM) manage the KV cache in non-contiguous memory blocks, or “pages,” which makes eviction and memory management much more flexible than a single, contiguous tensor.

All these strategies focus on *managing* the cache. Is it possible to just make the cache *smaller* from the start?



[source](#)

Can You Quantize the KV Cache Itself?

Quantization, or reducing the numerical precision of model parameters, is another powerful optimization. It is important to distinguish between two types.

Weight quantization techniques like AWQ (Activation-aware Weight Quantization) reduce the precision of the model’s *weights* (e.g., to 4-bit). This dramatically reduces

the overall model memory footprint, freeing up more VRAM for the KV cache, but it does *not* re

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Quantizing the KV cache *directly* (e.g., storing the K and V vectors in 8-bit or 4-bit precision) is a more advanced technique that directly tackles the cache bottleneck, though it can present accuracy challenges.

As we've seen, the KV Cache is not a simple "on/off" switch but a complex system of trade-offs.

What is the Central Trade-Off of KV Caching?

The KV Cache perfectly illustrates a fundamental engineering trade-off. It brilliantly solves the **quadratic compute problem** of autoregressive generation, making inference feasible. In doing so, it transforms the bottleneck into a **linear memory-bandwidth problem**. We trade raw computational load for a massive increase in memory consumption and data movement, which then becomes the new challenge to optimize.

So, looking back, what is the final verdict on this technique?

Why Does the KV Cache Remain a Critical Optimization?

Despite the memory challenges, the KV Cache is a non-negotiable, critical optimization for making LLM inference practical. It is the core mechanism that allows for responsive, real-time text generation. Without it, the latency of generating long-form text or engaging in a simple conversation would be prohibitively high, and the "magic" of modern AI would be lost.

Given its importance and its challenges, what's next for this technology?

In summary, the KV Cache is the unsung hero of fast LLM inference. By storing intermediate Key and Value states, it eliminates redundant computation and transforms an impossibly slow quadratic problem into a manageable memory-bandwidth challenge. We've explored its core mechanism, from the prefill and decode phases to its implementation in code, and delved into the advanced strategies like offloading, PagedAttention, and quantization that are used to manage its significant memory footprint. Understanding this trade-off is the key to building and deploying performant, large-scale language models.

If you found this deep dive helpful or have your own experiences with KV cache optimization, please share them in the comments. This helps us all improve our understanding of LLM inference.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

gagement

References:

KV Caching Explained: Optimizing Transformer Inference Efficiency A Blog post by Hafedh Hichri on Hugging Face huggingface.co	
Mastering LLM Techniques: Inference Optimization NVIDIA Technical Blog Stacking transformer layers to create large models results in better accuracies, few-shot learning capabilities, and... developer.nvidia.com	
Transformers Key-Value Caching Explained As the complexity of transformer-based models grow, so does the need to optimize their inference speed. KV caching is a... neptune.ai	
Understanding and Coding the KV Cache in LLMs from Scratch KV caches are one of the most critical techniques for efficient inference in LLMs in production. magazine.sebastianraschka.com	
How to Reduce KV Cache Bottlenecks with NVIDIA Dynamo NVIDIA Technical Blog As AI models grow larger and more sophisticated, inference, the process by which a model generates responses, is...	

developer.nvidia.com

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

Unlocking Longer Generation with Key-Value Cache Quantization

We're on a journey to advance and democratize artificial intelligence through open source and open science.

huggingface.co

KV Caching in LLM Inference A Comprehensive Review

Browse all previously published AI Tutorials here. I write everyday for my readers on actionable AI.

www.rohan-paul.com

AI

+

Artificial Intelligence

+

Deep Learning

+

LLM

+



Follow

Published in Towards AI

148K followers · Last published 1 hour ago

We train AI Engineers. We deploy Enterprise AI. Join 100K+ AI practitioners at Towards AI Academy or reach out for help making your company AI native. towardsai.com



Follow

Written by Burak Degirmencioglu

117 followers · 82 following

AI Engineer - Data Scientist | GenAI | LLM | Multi-Agent <https://www.linkedin.com/in/burakdegirmencioglu/>

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.

To make Medium work, we log user data. By using Medium, you agree to our Privacy Policy, including cookie policy.